AI-Powered Bug Tracker

Full-Stack Project Work Plan & Sprint Roadmap

Project Code	**AIBT-2026**
Document ID	**WORKPLAN-AIBT-001**
Project Type	**Full-Stack Web Application**
Start Date	**11 April 2026**
Project Manager	**Mr. Alok Ranjan | +91-91557 84760**
Version	**1.0 — Baseline**
Prepared by	**AI-Assisted Planning | April 2026**

TECHNOLOGY STACK

Layer	Technology
Frontend	React.js (Vite), React Router, Axios, CSS Modules
Backend	Java 17, Spring Boot 3, Spring Security (JWT), Spring Data JPA / Hibernate
Database	PostgreSQL 15
AI / Test Gen	Claude API / OpenAI / Gemini API + Playwright (Node.js)
Email	Brevo SMTP / Brevo Transactional API
Auth	JWT (access token + server-side blacklist for logout)
Build / Deploy	Maven (backend), npm (frontend), Docker-ready

USER ROLES IN SCOPE

Role	Key Responsibilities	Access Level
Admin	Manage users, create/edit/delete bugs, assign bugs, view all data	Full access

Developer	View assigned bugs, update status, view AI-generated test scripts	Assigned bugs only
Tester	Sign up, submit bugs, view AI tests & results, cancel & change password	Own bugs only

OUT OF SCOPE

Mobile applications (iOS / Android)
Multi-tenant or multi-project support
Real-time collaboration / WebSocket features
Payment gateway or billing integrations

PROJECT TIMELINE OVERVIEW

Sprint Roadmap at a Glance

PHASE 1	Phase — Backend	
S1	Foundation & Auth	Wk 1–2
S2	Bug CRUD APIs	Wk 2–3
S3	User Management APIs	Wk 3
S4	AI Integration	Wk 4
S5	Playwright Test Runner	Wk 5
S6	Email Notifications	Wk 5–6
S7	Testing & Hardening	Wk 6–7

PHASE 2	Phase — Frontend	
S8	Setup & Auth UI	Wk 7–8
S9	Admin Dashboard	Wk 8–9
S10	Developer Dashboard	Wk 9–10
S11	Tester Dashboard	Wk 10–11
S12	Polish & QA	Wk 11–12

Total estimated duration	**12 weeks (3 months)**
Total sprints	**12 sprints (7 backend + 5 frontend)**
Sprint cadence	**1–1.5 week sprints with review checkpoints**
Use case coverage	**UC-001 to UC-020 (all 20 use cases from URS-AIBT-001)**

PHASE 1 — BACKEND DEVELOPMENT

Spring Boot + PostgreSQL + AI + Playwright + Brevo

Sprint 1	Foundation & Auth	Week 1–2

Goal: Runnable Spring Boot app with JWT auth, all entities persisted

1.	Create Spring Boot 3 project via Spring Initializr (Web, JPA, Security, Validation, Lombok, PostgreSQL Driver)
2.	Configure application.properties: datasource, JPA DDL auto-create, server port
3.	Design and create JPA entities: User (id UUID, name, email, password, role ENUM, phone, createdAt), Bug (id UUID, title, description, severity ENUM, status ENUM, assignedTo FK, createdBy FK, createdAt, updatedAt), TestScript (id UUID, bugId FK, code TEXT, status ENUM, logs TEXT, executedAt)
4.	Implement Spring Security: SecurityFilterChain, JwtAuthFilter (OncePerRequestFilter), JwtUtil (generate / validate / extract claims)
5.	Token blacklist service for logout: in-memory Set or Redis — store invalidated JTIs
6.	Auth endpoints: POST /api/auth/signup (Tester only), POST /api/auth/login (all roles), POST /api/auth/logout
7.	Role-based route protection: ADMIN, DEVELOPER, TESTER roles mapped to endpoint patterns
8.	Global exception handler (@RestControllerAdvice): 401, 403, 404, 400 validation errors
9.	Unit tests for JwtUtil and AuthService

Sprint 2	Bug CRUD APIs	Week 2–3

Goal: Full bug lifecycle REST API with role-scoped access

1.	POST /api/bugs — create bug (Admin or Tester); validate mandatory fields (title, description, severity)
2.	GET /api/bugs — list all bugs (Admin); filter by assignedTo (Developer sees own); filter by createdBy (Tester sees own)
3.	GET /api/bugs/{id} — get single bug with associated TestScript
4.	PUT /api/bugs/{id} — full update (Admin only): title, description, severity, assignee, status
5.	DELETE /api/bugs/{id} — hard delete with confirmation (Admin only); cascade-delete linked TestScript
6.	PATCH /api/bugs/{id}/status — Developer updates status: Open / In Progress / Resolved / Closed
7.	PATCH /api/bugs/{id}/cancel — Tester withdraws own bug: sets status = Withdrawn
8.	Request/response DTOs for all endpoints; use MapStruct or manual mapping
9.	@Valid + custom constraint validators: severity enum, status transition rules
10.	Integration tests for all Bug endpoints using MockMvc

Sprint 3	User Management APIs	Week 3

Goal: Admin-controlled user CRUD + self-service password change

1.	GET /api/users — list all users (Admin): id, name, email, role, phone, createdAt
2.	PUT /api/users/{id} — Admin updates user: only email and phone are editable (name/role frozen)
3.	DELETE /api/users/{id} — Admin hard-deletes user; prevent self-deletion

4.	PATCH /api/users/password — Tester changes own password: validate currentPassword, newPassword, confirmPassword
5.	Password change validation: bcrypt verify current, enforce min-length, reject re-use of current
6.	Unit tests for UserService; integration tests for all User endpoints

PHASE 1 — BACKEND (continued)

Sprint 4	AI Integration	Week 4

Goal: Auto-generate Playwright test scripts on every bug submission

1.	AiService: build prompt from bug title + description; call Claude API (or Gemini/OpenAI) via RestTemplate / WebClient
2.	Parse AI response: extract raw Playwright JS code block; sanitise
3.	Save TestScript entity with code=, status=PENDING immediately after bug is saved
4.	Make AI call asynchronous (@Async + ThreadPoolTaskExecutor) so bug creation returns 201 instantly
5.	Handle AI API timeout (10s) and errors: mark TestScript.status = AI_FAILED; do not block bug save
6.	Configuration: AI provider URL, API key, model, temperature — all from application.properties / env vars
7.	Unit tests for AiService with mocked HTTP client

Sprint 5	Playwright Test Runner	Week 5

Goal: Execute AI-generated tests server-side and persist results

1.	PlaywrightRunnerService: write Playwright JS code to a temp file (/tmp/aibt/.spec.js)
2.	Execute via ProcessBuilder: ['node', tempFilePath] with 30s timeout
3.	Capture stdout + stderr; parse exit code: 0 = PASS, non-zero = FAIL
4.	Update TestScript: status=PASS/FAIL, logs=, executedAt=now()
5.	Chain execution immediately after AI generation completes (CompletableFuture chain)
6.	GET /api/bugs/{id}/test-result — returns TestScript (code, status, logs, executedAt) for frontend polling
7.	Ensure Node.js and Playwright are installed in the runtime environment (document setup steps)
8.	Integration test: submit dummy bug, verify TestScript status transitions

Sprint 6	Email Notifications (Brevo)	Week 5–6

Goal: Automated transactional emails for all key events

1.	EmailService: wrap Brevo SMTP or Brevo Transactional API (REST); inject credentials from env
2.	Email template 1 — Bug Created: to assigned Developer; includes bug title, severity, description excerpt, link
3.	Email template 2 — Bug Updated (Admin edit): to Developer + Tester; includes changed fields
4.	Email template 3 — Status Changed (Developer action): to Tester + Admin; new status highlighted
5.	Email template 4 — Bug Withdrawn (Tester cancel): to assigned Developer
6.	Email template 5 — Test Result: to Tester; PASS/FAIL badge, truncated logs
7.	All emails sent asynchronously (@Async) — do not block API response
8.	Integration test: mock Brevo API, verify correct recipients and subject lines per event

Sprint 7	Testing & Security Hardening	Week 6–7

Goal: Production-ready security, full test coverage, code review

1.	@PreAuthorize annotations on every controller method verified against role matrix

2.	JWT expiry, signature validation, and blacklist tested under adversarial inputs
3.	Input sanitisation: reject script tags and SQL patterns in free-text fields
4.	Rate limiting on /api/auth/login (e.g. 5 attempts / min via Bucket4j or custom filter)
5.	CORS configuration: whitelist React dev server and production domain only
6.	Full integration test suite: auth flow, bug CRUD, AI mock, email mock, test runner mock
7.	API documentation with SpringDoc OpenAPI (Swagger UI at /swagger-ui.html)
8.	Backend code review, refactor, and performance baseline check (response times under 200ms for CRUD)

PHASE 2 — FRONTEND DEVELOPMENT

React.js + Role-Based Dashboards + AI Test Viewer

Sprint 8	Setup & Auth UI	Week 7–8
Goal: Runnable React app with login, signup, and route guards		
1.	Vite + React 18 project scaffold; install React Router v6, Axios, and CSS Modules	
2.	Axios instance with base URL; request interceptor adds JWT Authorization header; response interceptor handles 401 redirect	
3.	JWT stored in memory (React context) — never localStorage; refresh via silent re-login or refresh token	
4.	Login page: email + password form, inline validation, error toast on failure (UC-001, UC-009, UC-014)	
5.	Signup page (Tester only): full name, email, password, confirm password, role=Tester hardcoded (UC-013)	
6.	ProtectedRoute component: checks auth context, redirects to /login if unauthenticated	
7.	RoleRoute component: checks user role, shows 403 page if wrong role	
8.	Logout flow: call POST /api/auth/logout, clear context, redirect to /login (UC-020)	
9.	Loading spinner and skeleton placeholder components for reuse	

Sprint 9	Admin Dashboard	Week 8–9
Goal: Complete admin control panel with bug and user management		
1.	Admin dashboard layout: sidebar nav (Bugs, Manage Users, Logout) + topbar with avatar	
2.	Bug list table: columns — ID, title, severity badge, status badge, assigned dev, created date, actions (edit/delete)	
3.	Table features: client-side sort by any column, filter by status / severity pill chips	
4.	Add Bug modal form: title, description (textarea), severity dropdown, assigned developer dropdown — POST /api/bugs (UC-003)	
5.	Edit Bug modal: pre-filled form from selected row — PUT /api/bugs/{id} (UC-004)	
6.	Delete Bug: confirmation dialog with bug title displayed — DELETE /api/bugs/{id} (UC-005)	
7.	Manage Users page: table of all users (name, email, role, phone) with Edit and Delete per row (UC-006)	
8.	Update User Profile modal: email + phone editable only — PUT /api/users/{id} (UC-007)	
9.	Delete User: confirmation dialog — DELETE /api/users/{id} (UC-008)	
10.	Toast notification system (success / error) on every API action	

PHASE 2 — FRONTEND (continued)

Sprint 10	Developer Dashboard	Week 9–10

Goal: Developer-focused bug view with status updates and AI script viewer

1.	Developer dashboard layout: sidebar (My Bugs, Logout) + topbar
2.	Assigned bugs table: title, severity, status, test result (PASS/FAIL/PENDING), filed date, actions
3.	Update Status: inline dropdown per row (Open / In Progress / Resolved / Closed) — PATCH /api/bugs/{id}/status (UC-011)
4.	Confirmation before status change: small popover with current → new status preview
5.	Email dispatch confirmation toast after status update
6.	Bug detail page (click any row): full bug info + AI Generated Test section (UC-012)
7.	AI test script viewer: syntax-highlighted code block (use highlight.js or Prism), Copy to clipboard button
8.	Test result section: PASS/FAIL badge, executedAt timestamp, scrollable execution log output
9.	Poll GET /api/bugs/{id}/test-result every 3s if status is PENDING; stop on PASS or FAIL
10.	Logout flow: UC-020

Sprint 11	Tester Dashboard	Week 10–11

Goal: Core tester UX — submit bugs, track AI tests, manage account

1.	Tester dashboard: sidebar (Dashboard, My Bugs, Test Results, Logout) + stat cards (total filed, open/in-progress count, AI tests generated, pass rate)
2.	My bugs table: title, severity, status, test result, filed date, View + Cancel actions
3.	Filter pills: All / Open / In Progress / Resolved — client-side filter (UC-015)
4.	Submit Bug modal: title, plain-English description (AI uses this), severity, assign to developer — POST /api/bugs (UC-016)
5.	AI generation feedback: after submission show 'AI is generating test...' spinner; auto-refresh test result
6.	Cancel Bug: confirmation dialog, PATCH /api/bugs/{id}/cancel sets status = Withdrawn; Cancel button disabled for Resolved/Withdrawn rows (UC-018)
7.	Bug detail page: AI test script viewer + test result logs (same component as Developer, read-only) (UC-017)
8.	Change Password modal: current password, new password, confirm new — PATCH /api/users/password (UC-019)
9.	Inline validation on all forms (required, min-length, email format, password match)

Sprint 12	Polish, QA & Deployment Prep	Week 11–12

Goal: Production-grade UX, cross-browser QA, deployment checklist

1.	Responsive layout audit: ensure all dashboards work at 1280px, 1024px minimum
2.	Dark mode support: CSS variables / media query implementation
3.	Empty state illustrations for tables with zero results
4.	Error boundary components: catch render errors, show friendly fallback UI
5.	Accessibility pass: ARIA labels on all interactive elements, keyboard navigation for modals

6.	Cross-browser test: Chrome, Firefox, Edge — verify JWT flow, file upload if any
7.	End-to-end smoke test: Tester signup → submit bug → AI test generated → Developer sees bug → status update → Tester sees result
8.	Environment config: .env.production (API base URL, AI key placeholder), README with setup steps
9.	Dockerfile for frontend (nginx), docker-compose linking backend + frontend + PostgreSQL
10.	Final team review, bug bash, and sign-off against all 20 use cases (UC-001 to UC-020)

TEAM AGREEMENTS & DEFINITION OF DONE

Rules We Stick To — Every Sprint, Every Task

Definition of Done (DoD)

A task is only marked complete when ALL of the following are true:

1	Code is committed to the correct branch (feature/AIBT-)
2	Unit or integration test written and passing for the change
3	No compiler warnings; checkstyle / ESLint passes with 0 errors
4	API endpoint tested manually via Swagger UI or Postman
5	Relevant use case ID noted in the commit message (e.g. UC-003)
6	No hardcoded credentials, API keys, or environment-specific URLs in source code
7	Email notification verified (real send or mock assertion) if the task touches notifications
8	PR reviewed and approved by at least one other team member before merge

Git Branch Strategy

Branch	Purpose	Merge Target
main	Production-ready code only	—
develop	Integration branch; all features merge here first	main (release)
feature/AIBT-	One branch per task/use case	develop
hotfix/	Critical production fixes	main + develop

Use Case Coverage Matrix

Every use case from URS-AIBT-001 is covered by a specific sprint task.

UC	Use Case Name	Sprint	Role
UC-001	Admin Login	S1 + S8	Admin
UC-002	Admin Dashboard	S9	Admin
UC-003	Add Bug	S2 + S9	Admin
UC-004	Update Bug Detail	S2 + S9	Admin
UC-005	Delete Bug	S2 + S9	Admin
UC-006	Manage Users	S3 + S9	Admin
UC-007	Update User Profile	S3 + S9	Admin
UC-008	Delete User	S3 + S9	Admin

UC-009	Developer Login	S1 + S8	Developer
UC-010	Developer Dashboard	S10	Developer
UC-011	Update Bug Status	S2 + S10	Developer
UC-012	View AI-Generated Test	S4 + S10	Developer
UC-013	Tester Signup	S1 + S8	Tester
UC-014	Tester Login	S1 + S8	Tester
UC-015	Tester Dashboard	S11	Tester
UC-016	Submit Bug (Core)	S2+S4+S11	Tester
UC-017	View Test Result	S5 + S11	Tester
UC-018	Cancel Bug Report	S2 + S11	Tester
UC-019	Change Password	S3 + S11	Tester
UC-020	Logout (All Roles)	S1 + S8	All

API ENDPOINTS QUICK REFERENCE

Backend REST API — Spring Boot

Auth Endpoints

POST	/api/auth/signup	Register new Tester	Public
POST	/api/auth/login	Login (all roles) — returns JWT	Public
POST	/api/auth/logout	Invalidate JWT token	All roles

Bug Endpoints

POST	/api/bugs	Create bug (triggers AI + email)	Admin, Tester
GET	/api/bugs	List bugs (scoped by role)	All roles
GET	/api/bugs/{id}	Get single bug + test script	All roles
PUT	/api/bugs/{id}	Full update of bug	Admin
DELETE	/api/bugs/{id}	Hard delete bug + test script	Admin
PATCH	/api/bugs/{id}/status	Update resolution status	Developer
PATCH	/api/bugs/{id}/cancel	Withdraw bug (status=Withdrawn)	Tester
GET	/api/bugs/{id}/test-result	Get AI test script + execution result	All roles

User Endpoints

GET	/api/users	List all users	Admin
PUT	/api/users/{id}	Update user email + phone	Admin
DELETE	/api/users/{id}	Delete user account	Admin
PATCH	/api/users/password	Change own password	Tester

Core JPA Entities

User	id (UUID PK), name, email (unique), password (bcrypt), role (ADMIN/DEVELOPER/TESTER), phone, createdAt
Bug	id (UUID PK), title, description (TEXT), severity (CRITICAL/HIGH/MEDIUM/LOW), status (OPEN/IN_PROGRESS/RESOLVED/CLOSED/WITHDRAWN), assignedTo (FK User), createdBy (FK User), createdAt, updatedAt
TestScript	id (UUID PK), bugId (FK Bug, unique), code (TEXT), status (PENDING/PASS/FAIL/AI_FAILED), logs (TEXT), executedAt

Risk Register

Risk	Likelihood	Impact	Mitigation
AI API unavailable / slow	Medium	High	Async call with 10s timeout; mark TestScript AI_FAILED; bug still saved
Playwright not installed on server	Low	High	Document Node.js + Playwright setup in README; include in Dockerfile
JWT token leakage	Low	Critical	Short expiry (1h); server-side blacklist; HTTPS only in production
Email delivery failure (Brevo)	Low	Medium	Async email; log failures; retry queue; do not block bug API response
Scope creep on AI features	High	Medium	Strictly limit AI to test generation only; all other features are manual
PostgreSQL schema drift	Medium	High	Use Flyway or Liquibase for schema migrations from Sprint 1 onward